WIKIPEDIA 25
25 years of the free encyclopedia

Transformer (deep learning)

In deep learning, the **transformer** is a family of artificial neural network architectures based on the multi-head attention mechanism, in which input data such as text, images, or audio is converted to a sequence of numerical representations called tokens, and each token is converted into a vector through an embedding layer.[1] At each layer, each token is then contextualized within the scope of the context window with other (unmasked) tokens via a parallel multi-head attention mechanism, allowing the signal for key tokens to be amplified and less important tokens to be diminished. Because self-attention alone is permutation-invariant, transformers inject positional information, typically through positional encodings or learned positional embeddings, so token order can affect the output.[1]



A standard transformer architecture, showing on the left an encoder, and on the right a decoder. Note: this uses the pre-LN convention, which is different from the post-LN convention used in the original 2017 transformer.

Because transformers do not process tokens one at a time, their computations can be parallelized across sequence positions during training more readily than those of recurrent neural networks like long short-term memory (LSTM).[2] Later variations have been widely adopted for training large language models (LLMs) on large (language) datasets.

The original version of the transformer architecture was proposed in the 2017 paper "Attention Is All You Need" by researchers at Google.[1] The predecessors of transformers were developed as an improvement over previous architectures for machine translation,[3][4] but have found many applications since. They are used in large-scale natural language processing, computer vision (vision transformers), reinforcement learning,[5] audio,[6] multimodal learning, robotics,[7] and playing chess.[8] It has also led to the development of pre-trained systems, such as generative pre-trained transformers (GPTs).[9]

History

Predecessors

Before transformers, recurrent neural networks (RNNs) were widely used for sequence modelling and generation. A well-cited early example was the Elman network (1990). In theory, the information from one token can propagate arbitrarily far down the sequence, but in practice the vanishing-gradient problem leaves the model's state at the end of a long sentence without precise, extractable information about preceding tokens.

A key breakthrough was LSTM (originally described in a 1995 technical report and formally published in 1997),[2][note 1] an RNN that introduced gating mechanisms to mitigate the vanishing gradient problem, allowing efficient learning of long-sequence modelling. One key architectural element was the use of *multiplicative gating units*, in which the outputs of some neurons modulate the outputs of others. These multiplicative units are conceptually distinct from the additive attention mechanism later introduced for sequence-to-sequence models. [10] Neural networks using multiplicative units were later called *sigma-pi networks*[11] or *higher-order networks*.[12] LSTM became the standard architecture for long sequence modelling until the 2017 publication of transformers. However, LSTM still used sequential processing, like most other RNNs.[note 2] Specifically, RNNs operate one token at a time from first to last; they cannot operate in parallel over all tokens in a sequence.

Transformers allow parallel processing of tokens during training, but standard self-attention has computational and memory costs that grow quadratically with sequence length. The fast weight controller, proposed in 1992, used one network to generate input-dependent weights for another network. This method built on earlier work on "fast weights" and "dynamic links".[13][14][15] A slow neural network learns by gradient descent to generate keys and values for computing the weight changes of the fast neural network which computes answers to queries.[16] This was later shown to be equivalent to the unnormalized linear transformer.[17][18]

Attention with seq2seq

The idea of encoder–decoder sequence transduction had been developed in the early 2010s; commonly cited as the originators that produced seq2seq are two concurrently published papers from 2014.[19][20]

A 380M-parameter model for machine translation uses two long short-term memories (LSTM).[20] Its architecture consists of two parts. The *encoder* is an LSTM that takes in a sequence of tokens and turns it into a vector. The *decoder* is another LSTM that converts the vector into a sequence of tokens. Similarly, another 130M-parameter model used gated recurrent units (GRU) instead of LSTM.[19] Later research showed that GRUs are neither better nor worse than LSTMs for seq2seq.[21][22]

These early seq2seq models had no attention mechanism, and the state vector is accessible only after the *last* word of the source text was processed. Although in theory such a vector retains the information about the whole original sentence, in practice the information is poorly preserved. This is because the input is processed sequentially by one recurrent network into a *fixed*-size output vector, which is then processed by another recurrent network into an output. If the input is long, then the output vector would not be able to contain all relevant information, degrading the output. As evidence, reversing the input sentence improved seq2seq translation.[23]

The *RNN search* model introduced an attention mechanism to seq2seq for machine translation to solve the bottleneck problem (of the *fixed-size* output vector), allowing the model to process long-distance dependencies more easily. The name is because it "emulates searching through a source sentence during decoding a translation".[3]

The relative performances were compared between global (that of *RNN search*) and local (sliding window) attention model architectures for machine translation, finding that mixed attention had higher quality than global attention, while local attention reduced translation time.[24]

In 2016, Google Translate was revamped to Google Neural Machine Translation, which replaced the previous model based on statistical machine translation. The new model was a seq2seq model where the encoder and the decoder were both 8 layers of bidirectional LSTM.[25] It took nine months to develop, and it outperformed the statistical approach, which took ten years to develop.[26]

Parallelizing attention

Seq2seq models with attention (including self-attention) still suffered from the same issue with recurrent networks, which is that they are hard to parallelize, which prevented them from being accelerated on GPUs. In 2016, *decomposable attention* applied a self-attention mechanism to feedforward networks, which are easy to parallelize, and achieved state-of-the-art (SOTA) results in textual entailment with an order of magnitude fewer parameters than LSTMs.[27] One of its authors, Jakob Uszkoreit, suspected that attention *without* recurrence would be sufficient for language translation, thus the title "attention is *All* you need".[28] That hypothesis was against conventional wisdom at the time, and even his father Hans Uszkoreit, a well-known computational linguist, was skeptical.[28] In the same year, self-attention (called *intra-attention or intra-sentence attention*) was proposed for LSTMs.[29]

On 2017-06-12, the original (100M-parameter) encoder–decoder transformer model was published in the "Attention is All you need" paper. At the time, the focus of the research was on improving seq2seq for machine translation, by removing its recurrence to process all tokens in parallel, but preserving its dot-product attention mechanism to keep its text processing performance.[1] This led to the introduction of a multi-head attention model that was easier to parallelize due to the use of independent heads and the lack of recurrence. Its parallelizability was an important factor to its widespread use in large neural networks.[30]

AI boom era

In spring 2017, before the "Attention is All you need" preprint was published, one of the co-authors applied the "decoder-only" variation of the architecture to generate fictitious Wikipedia articles.[31] Transformer architecture is now used alongside many generative models that contribute to the ongoing AI boom.

The "reference implementation" of the original Transformer was written in a TensorFlow library.[32][33] In language modelling, ELMo (2018) was a bi-directional LSTM that produces contextualized word embeddings, improving upon the line of research from bag of words and word2vec. It was followed by BERT (2018), an encoder-only transformer model.[34] In October 2019, Google started using BERT to process search queries.[35] In 2020, Google Translate replaced the previous RNN-encoder–RNN-decoder model by a transformer-encoder–RNN-decoder model.[36]

Starting in 2018, the OpenAI GPT series of decoder-only transformers became state of the art in natural language generation. At the end of 2022, ChatGPT, a chatbot based on a fine-tuned variant of GPT-3.5, became unexpectedly[37][38] popular, triggering a boom around large language models.[39]

Transformers have been applied in modalities beyond text. Four days after the publication of "Attention is All You Need", a multimodal transformer architecture, MultiModel, was published by most authors of that paper.[40] Other examples include the vision transformer, which has led to developments in convolutional neural networks.[41] Image and video generators like DALL-E (2021), Stable Diffusion 3

(2024), and Sora (2024), use transformers to analyse input data (like text prompts) by breaking it down into "tokens" and then calculating the relevance between each token using self-attention, which helps the model simulate the context and relationships within the data.

Training

Methods for stabilizing training

The plain transformer architecture had difficulty in converging. In the original paper,[1] the authors recommended using learning rate warmup. That is, the learning rate should linearly scale up from 0 to maximal value for the first part of the training (usually recommended to be 2% of the total number of training steps), before decaying again.

A 2020 paper found that using layer normalization *before* (instead of after) multihead attention and feedforward layers stabilizes training, not requiring learning rate warmup.[42] This is the "pre-LN Transformer" and is more commonly used, compared to the original "post-LN Transformer".

Regularization

The original transformer used dropout during training to reduce overfitting. Dropout was applied to the output of each sublayer before its residual connection and normalization, as well as to the sums of token embeddings and positional encodings. The original paper also used label smoothing, which replaces a certain target distribution with one that assigns a small amount of probability to alternative tokens. These techniques can improve generalization by going against overconfidence in the training targets.[43]

Distributed Training

Large transformer models are typically trained across multiple GPUs or other accelerators because their parameters, activations, and training data may not fit on a single device. Data parallelism copies the model across devices and splits different batches of training examples among them; gradients are then synchronized before parameters are updated. Model parallelism divides a model itself across devices. Tensor parallelism splits operations within individual layers, while pipeline parallelism assigns consecutive groups of layers to different devices.[44]

Pretrain-finetune

Transformers typically are first pretrained by self-supervised learning on a large generic dataset, followed by supervised fine-tuning on a small task-specific dataset. The pretrain dataset is typically an unlabeled large corpus, such as The Pile. Tasks for pretraining and fine-tuning commonly include:

- reading comprehension
- sentiment analysis[1]
- paraphrasing[1]

The T5 transformer report[45] documents a large number of natural language pretraining tasks. Some examples are:

- restoring or repairing incomplete or corrupted text. For example, the input, *"Thank you ~~ me to your party ~~ week",* might generate the output, *"Thank you **for inviting** me to your party **last** week".*
- translation between natural languages (machine translation)
- judging the pragmatic acceptability of natural language. For example, the following sentence might be judged "not acceptable",[46] because even though it is syntactically well-formed, it is improbable in ordinary human usage: *The course is jumping well.*

While each of these tasks is trivial or obvious for human native speakers of the language (or languages), they have typically proved challenging for previous generations of machine learning architecture.

Knowledge distillation

Knowledge distillation is a method for training a smaller model, called a "student", to reproduce the output behavior of a larger "teacher" model. In transformer models, the student can be trained using the teacher's output probability distribution, intermediate representations, or attention patterns in addition to the original training labels. Distillation can lower computational and memory requirements of deployment while keeping most of the teacher model's performance on a specified task.

Tasks

In general, there are three classes of language modelling tasks: "masked",[47] "autoregressive",[48] and "prefixLM".[49] These classes are independent of a specific modeling architecture such as transformer, but they are often discussed in the context of transformer.

In a masked task,[47] one or more of the tokens is masked out, and the model would produce a probability distribution predicting what the masked-out tokens are based on the context. The loss function for the task is typically sum of log-perplexities for the masked-out tokens:

$$\text{Loss} = -\sum_{t \in \text{masked tokens}} \ln(\text{probability of } t \text{ conditional on its context})$$

and the model is trained to minimize this loss function. The BERT series of models are trained for masked token prediction and another task. ("Masked" as in "masked language modelling" is not "masked" as in "masked attention".)

In an autoregressive task,[48] the entire sequence is masked at first, and the model produces a probability distribution for the first token. Then the first token is revealed and the model predicts the second token, and so on. The loss function for the task is still typically the same. The GPT series of models are trained by autoregressive tasks.

In a prefixLM task,[49] the sequence is divided into two parts. The first part is presented as context, and the model predicts the first token of the second part. Then that would be revealed, and the model predicts the second token, and so on. The loss function for the task is still typically the same. The T5 series of models are trained by prefixLM tasks. ("PrefixLM" as in "prefix language modeling" is not "prefixLM" as in "prefix language model".)

Architecture

All transformers have the same primary components:

- Tokenizers, which convert text into tokens.
- Embedding layer, which converts tokens and positions of the tokens into vector representations.
- Transformer layers, which carry out repeated transformations on the vector representations, extracting more and more linguistic information. These consist of alternating attention and feedforward layers. There are two major types of transformer layers: encoder layers and decoder layers, with further variants.
- Un-embedding layer, which converts the final vector representations back to a probability distribution over the tokens.

The following description follows exactly the transformer as described in the original paper. There are variants, described in the following section.

By convention, we write all vectors as row vectors. For example, pushing a vector through a linear layer means multiplying it by a weight matrix on the right, as $\boldsymbol{x}\boldsymbol{W}$.

Tokenization

As the transformer architecture natively consists of operations over numbers (matrix multiplications, dot products, activation functions) rather than over text, there must first be a mapping from any input text to some numerical representation. A text is cleaned, segmented into subsections, then each subsection is mapped to an integer according to a lookup table (also called the *vocabulary* $\boldsymbol{V}$). Subsections that do not appear in the vocabulary are usually represented by a special token `[UNK]` ("unknown").

The vocabulary $\boldsymbol{V}$ itself needs to be constructed before tokenization. This construction step is called vocabularization. The vocabulary can either be learned on a given textual corpus, or downloaded without learning on any textual corpus.

Commonly used algorithms include byte pair encoding (BPE) and the unigram language model (ULM), each of which include a vocabularization algorithm and a tokenization algorithm. If a vocabulary is already known, then only tokenization needs to be done. An example tokenization algorithm is greedily recognising tokens left-to-right.

Embedding

Each integer token identifier is converted into an embedding vector via a lookup table. Equivalently stated, it multiplies a one-hot representation of the token identifier by an embedding matrix $\boldsymbol{M}$. For example, if the input token's identifier is $\mathbf{3}$, then the one-hot representation is $[\mathbf{0}, \mathbf{0}, \mathbf{0}, \mathbf{1}, \mathbf{0}, \mathbf{0}, \ldots]$, and its embedding vector is

$$\mathbf{Embed}(\mathbf{3}) = [\mathbf{0}, \mathbf{0}, \mathbf{0}, \mathbf{1}, \mathbf{0}, \mathbf{0}, \ldots]\boldsymbol{M}$$

The token embedding vectors are added to their respective positional encoding vectors (see below), producing the sequence of input vectors.

The dimension of an embedding vector is called *hidden size* or *embedding size* and written as d_{emb}.[34] This size is written as d_{model} in the original transformer paper.[1]

Un-embedding

An un-embedding layer is almost the reverse of an embedding layer. Whereas an embedding layer converts a token identifier into a vector, an un-embedding layer converts a vector into a probability distribution over tokens.

The un-embedding layer is a linear-softmax layer:

$$\text{UnEmbed}(x) = \text{softmax}(xW + b)$$

The matrix has shape $(d_{\text{emb}}, |V|)$. Some architectures use the transpose of the embedding matrix M as the un-embedding matrix W in order to avoid needing double the amount of embedding-related parameters and to avoid divergence during training. This practice is called *weight tying*.

Positional encoding

A positional encoding is a fixed-size vector representation of the relative positions of tokens within a sequence: it provides the transformer model with information about *where* the words are in the input sequence. This induces a bias towards the order of the input sequence, so that, for example, the input sequence "man bites dog" is processed differently from "dog bites man".

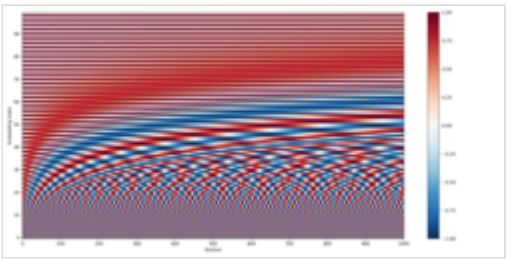


Illustration of (absolute) positional encoding with parameters $N = 10000, d = 100$

The positional encoding is defined as a function of type $f : \mathbb{R} \to \mathbb{R}^d$, where d is a positive even integer. The full positional encoding defined in the original paper is:

$$(f(t)_{2k}, f(t)_{2k+1}) = (\sin(\theta), \cos(\theta)) \quad \forall k \in \{0, 1, \ldots, d/2 - 1\}$$

where $\theta = \frac{t}{r^k}, r = N^{2/d}$.[1]

Here, N is a free parameter that should be significantly larger than the biggest k that would be input into the positional encoding function. The original paper uses $N = 10000$.

The function is in a simpler form when written as a complex function of type $f : \mathbb{R} \to \mathbb{C}^{d/2}$

$$f(t) = \left(e^{it/r^k}\right)_{k=0,1,\ldots,\frac{d}{2}-1}$$

where $r = N^{2/d}$.

The main reason for using this positional encoding function is that using it, shifts are linear transformations:

$$f(t + \Delta t) = \mathrm{diag}(f(\Delta t)) f(t)$$

where $\Delta t \in \mathbb{R}$ is the distance one wishes to shift. This allows the transformer to take any encoded position, and find the encoding of the position n-steps-ahead or n-steps-behind, by a matrix multiplication.

By taking a linear sum, any convolution can also be implemented as linear transformations:

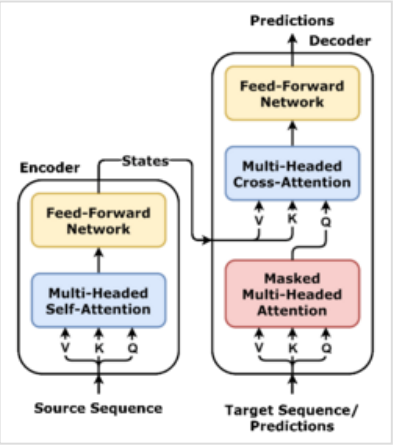
$$\sum_j c_j f(t + \Delta t_j) = \left(\sum_j c_j \, \mathrm{diag}(f(\Delta t_j)) \right) f(t)$$

for any constants c_j. This allows the transformer to take any encoded position and find a linear sum of the encoded locations of its neighbors. This sum of encoded positions, when fed into the attention mechanism, would create attention weights on its neighbors, much like what happens in a convolutional neural network language model. In the author's words, "we hypothesized it would allow the model to easily learn to attend by relative position."

In typical implementations, all operations are done over the real numbers, not the complex numbers, but since complex multiplication can be implemented as real 2-by-2 matrix multiplication, this is a mere notational difference.

Encoder–decoder (overview)

Like earlier seq2seq models, the original transformer model used an **encoder–decoder** architecture. The encoder consists of encoding layers that process all the input tokens together one layer after another, while the decoder consists of decoding layers that iteratively process the encoder's output and the decoder's output tokens so far.



One encoder–decoder block

The purpose of each encoder layer is to create contextualized representations of the tokens, where each representation corresponds to a token that "mixes" information from other input tokens via self-attention mechanism. Each decoder layer contains two attention sublayers: (1) cross-attention for incorporating the output of encoder (contextualized input token representations), and (2) self-attention for "mixing" information among the input tokens to the decoder (i.e. the tokens generated so far during inference time).

Both the encoder and decoder layers have a feed-forward neural network for additional processing of their outputs and contain residual connections and layer normalization steps. These feed-forward layers contain most of the parameters in a transformer model.

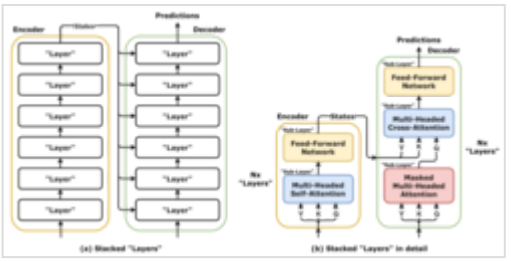
Feedforward network

The feedforward network (FFN) modules in a transformer are 2-layered multilayer perceptrons:

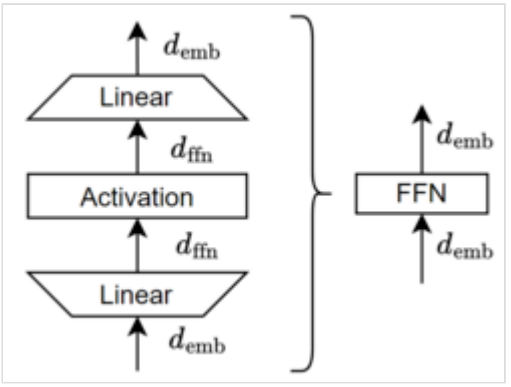
$$\mathbf{FFN}(x) = \phi(xW^{(1)} + b^{(1)})W^{(2)} + b^{(2)}$$

where $W^{(1)}$ and $W^{(2)}$ are weight matrices and $b^{(1)}$ and $b^{(2)}$ are bias vectors, and ϕ is its activation function. The original transformer used ReLU activation.

The number of neurons in the middle layer is called *intermediate size* (GPT),[50] *filter size* (BERT),[34] or *feedforward size* (BERT).[34] It is typically larger than the embedding size. For example, in both GPT-2 series and BERT series, the intermediate size of a model is 4 times its embedding size: $d_{\text{ffn}} = 4d_{\text{emb}}$.



A transformer is composed of stacked encoder layers and decoder layers.



The feedforward network module. It is a two-layered network that maps d_{emb}-dimensional vectors into d_{emb}-dimensional vectors.

Scaled dot-product attention

Attention head

The attention mechanism used in the transformer architecture are scaled dot-product attention units. For each unit, the transformer model learns three weight matrices: the query weights W^Q, the key weights W^K, and the value weights W^V.

The module takes three sequences, a query sequence, a key sequence, and a value sequence. The query sequence is a sequence of length $\ell_{\text{seq, query}}$, and each entry is a vector of dimension $d_{\text{emb, query}}$. Similarly for the key and value sequences.

For each vector $x_{i,\text{query}}$ in the query sequence, it is multiplied by a matrix W^Q to produce a query vector $q_i = x_{i,\text{query}}W^Q$. The matrix of all query vectors is the query matrix:

$$Q = X_{\text{query}}W^Q$$

Similarly, we construct the key matrix $K = X_{\text{key}}W^K$ and the value matrix $V = X_{\text{value}}W^V$.

It is usually the case that all W^Q, W^K, W^V are square matrices, meaning $d_{\text{emb, query}} = d_{\text{query}}$, etc.

Attention weights are calculated using the query and key vectors: the attention weight a_{ij} from token i to token j is the dot product between q_i and k_j. The attention weights are divided by the square root of the dimension of the key vectors, $\sqrt{d_k}$, which stabilizes gradients during training, and passed through a softmax which normalizes the weights. The fact that W^Q and W^K are different matrices allows attention

to be non-symmetric: if token i attends to token j (i.e. $q_i \cdot k_j$ is large), this does not necessarily mean that token j will attend to token i (i.e. $q_j \cdot k_i$ could be small). The output of the attention unit for token i is the weighted sum of the value vectors of all tokens, weighted by a_{ij}, the attention from token i to each token.

The attention calculation for all tokens can be expressed as one large matrix calculation using the softmax function, which is useful for training due to computational matrix operation optimizations that quickly compute matrix operations. The matrices Q, K and V are defined as the matrices where the ith rows are vectors q_i, k_i, and v_i respectively. Then we can represent the attention as

$$\mathbf{Attention}(Q, K, V) = \mathbf{softmax}\left(\frac{QK^{\mathrm{T}}}{\sqrt{d_k}}\right)V$$

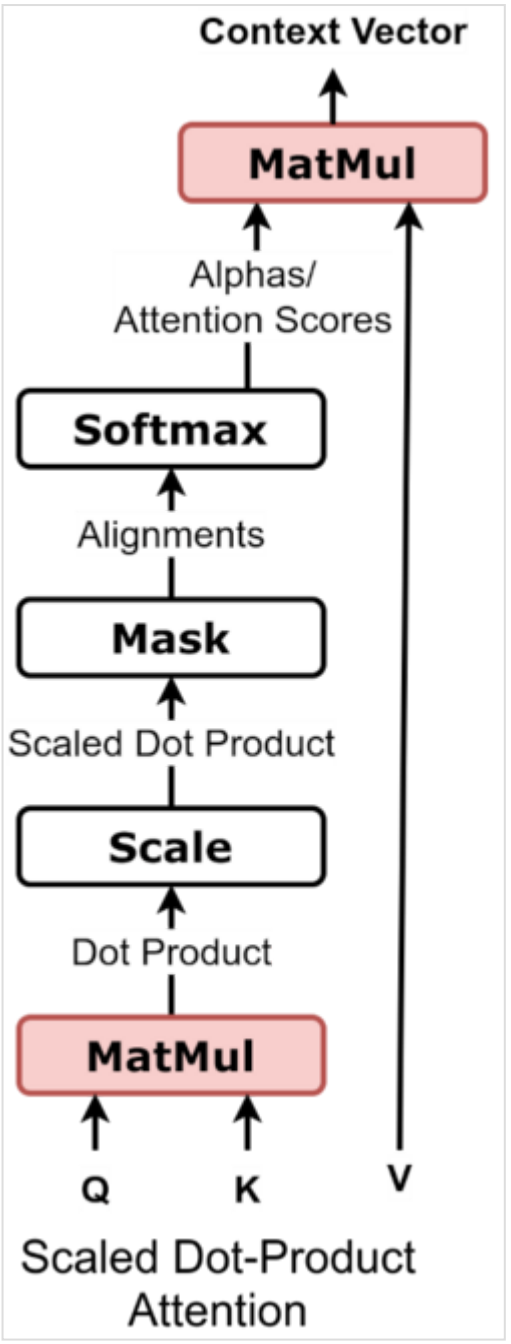
where the softmax is applied over each of the rows of the matrix.

The number of dimensions in a query vector is *query size* d_{query} and similarly for the *key size* d_{key} and *value size* d_{value}. The output dimension of an attention head is its *head dimension* d_{head}. The attention mechanism requires the following three equalities to hold:

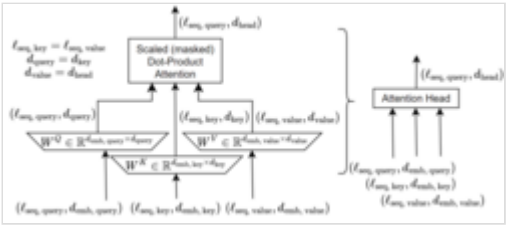
$$\ell_{\text{seq, key}} = \ell_{\text{seq, value}}, \; d_{\text{query}} = d_{\text{key}}, \; d_{\text{value}} = d_{\text{head}}$$

but is otherwise unconstrained.

If the attention head is used in a self-attention fashion, then $X_{\text{query}} = X_{\text{key}} = X_{\text{value}}$. If the attention head is used in a cross-attention fashion, then usually $X_{\text{query}} \neq X_{\text{key}} = X_{\text{value}}$. It is theoretically possible for all three to be different, but that is rarely the case in practice.



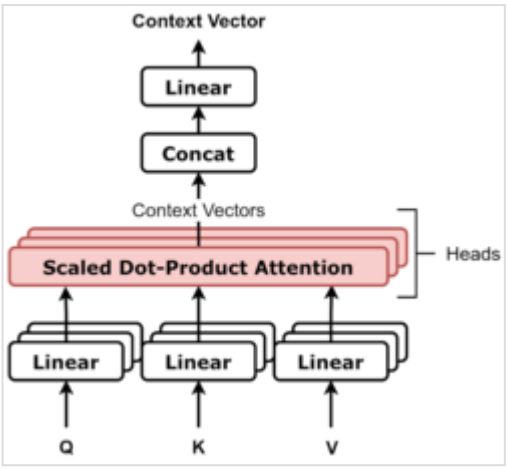
Scaled dot-product attention, block diagram


Exact dimension counts within an attention head module

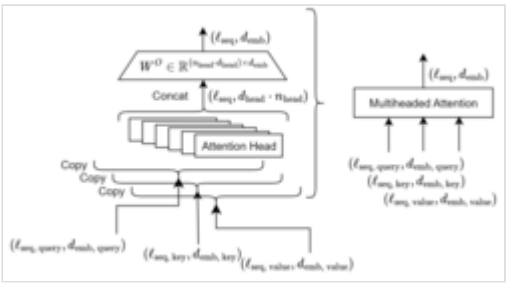
Multihead attention

One set of $\left(W^Q, W^K, W^V\right)$ matrices is called an *attention head*, and each layer in a transformer model has multiple attention heads. While each attention head attends to the tokens that are relevant to each token, multiple attention heads allow the model to do this for different definitions of "relevance". Specifically, the query and key projection matrices, W^Q and W^K, which are involved in the attention score computation, defines the "relevance". Meanwhile, the value projection matrix W^V, in

combination with the part of the output projection matrix W^O, determines how the attended tokens influence what information is passed to subsequent layers and ultimately the output logits. In addition, the scope of attention, or the range of token relationships captured by each attention head, can expand as tokens pass through successive layers. This allows the model to capture more complex and long-range dependencies in deeper layers. Many transformer attention heads encode relevance relations that are meaningful to humans. For example, some attention heads can attend mostly to the next word, while others mainly attend from verbs to their direct objects.[51] The computations for each attention head can be performed in parallel, which allows for fast processing. The outputs for the attention layer are concatenated to pass into the feedforward neural network layers.



Multihead attention, block diagram



Exact dimension counts within a multihead attention module

Concretely, let the multiple attention heads be indexed by i, then we have

$$\text{MultiheadAttention}(Q, K, V) = \text{Concat}_{i \in [n_{\text{heads}}]}(\text{At}$$

where the matrix X is the concatenation of word embeddings, and the matrices $W^{Q,i}, W^{K,i}, W^{V,i}$ are "projection matrices" owned by individual attention head i, and W^O is a final projection matrix owned by the whole multihead attention module.

It is theoretically possible for each attention head to have a different head dimension d_{head}, but that is rarely the case in practice.

As an example, in the smallest GPT-2 model, there are only self-attention mechanisms. It has the following dimensions:

$$d_{\text{emb}} = 768, n_{\text{head}} = 12, d_{\text{head}} = 64$$

Since $12 \times 64 = 768$, its output projection matrix $W^O \in \mathbb{R}^{(12\times 64)\times 768}$ is a square matrix.

Masked attention

The transformer architecture is constructed to calculate output tokens iteratively. Assuming $t = 0$ refers to the calculation of the first output token $i = 0$, for step $t > 0$, the output token $i = 0$ shall remain constant. This ensures properties of the model similar to autoregressive models.[1] Therefore, at every time step t, the calculation for all outputs i should not have access to tokens at position j for $j >= i$ (as it naturally is the case for time step $t = i$, when tokens $j > t$ are not yet calculated). This behavior may be accomplished before the softmax stage by adding a mask matrix M that is $-\infty$ at entries where the attention link must be cut, and 0 at other places:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathrm{T}}}{\sqrt{d_k}} + M\right)V$$

The following matrix is commonly used in decoder self-attention modules, called "causal masking":

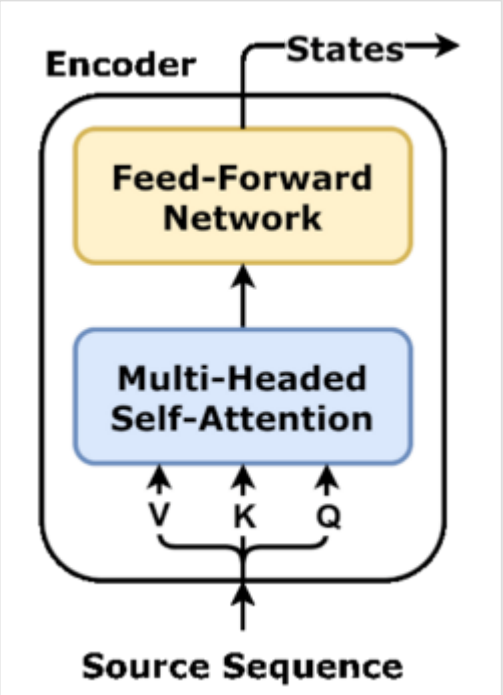
$$M_{\text{causal}} = \begin{bmatrix} 0 & -\infty & -\infty & \dots & -\infty \\ 0 & 0 & -\infty & \dots & -\infty \\ 0 & 0 & 0 & \dots & -\infty \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 0 \end{bmatrix}$$

In words, it means that each token can pay attention to itself, and every token before it, but not any after it. A non-masked attention module can be thought of as a masked attention module where the mask has all entries zero. As an example of an uncommon use of mask matrix, the XLNet considers all masks of the form $PM_{\text{causal}}P^{-1}$, where P is a random permutation matrix.[52]

Encoder

An encoder consists of an embedding layer, followed by multiple encoder layers.

Each encoder layer consists of two major components: a self-attention mechanism and a feed-forward layer. It takes an input as a sequence of input vectors, applies the self-attention mechanism, to produce an intermediate sequence of vectors,



One encoder layer

then applies the feed-forward layer for each vector individually. Schematically, we have:

$$\begin{aligned}
&\text{given input vectors } h_0, h_1, \dots \\
&\text{combine them into a matrix } H = \begin{bmatrix} h_0 \\ h_1 \\ \vdots \end{bmatrix} \\
&\mathrm{EncoderLayer}(H) = \begin{bmatrix} \mathrm{FFN}(\mathrm{MultiheadAttention}(H, H, H)_0) \\ \mathrm{FFN}(\mathrm{MultiheadAttention}(H, H, H)_1) \\ \vdots \end{bmatrix}
\end{aligned}$$

where $\mathbf{FFN}$ stands for "feed-forward network". We can more succinctly write it as

$$\mathrm{EncoderLayer}(H) = \mathrm{FFN}(\mathrm{MultiheadAttention}(H, H, H))$$

with the implicit convention that the $\mathbf{FFN}$ is applied to each row of the matrix individually.

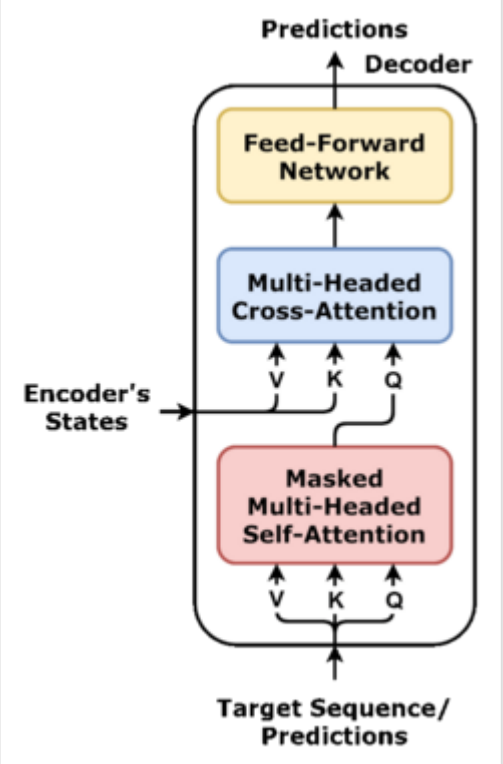
The encoder layers are stacked. The first encoder layer takes the sequence of input vectors from the embedding layer, producing a sequence of vectors. This sequence of vectors is processed by the second encoder, and so on. The output from the final encoder layer is then used by the decoder.

As the encoder processes the entire input all at once, every token can attend to every other token (all-to-all attention), so there is no need for causal masking.

Decoder

A decoder consists of an embedding layer, followed by multiple decoder layers, followed by an un-embedding layer.

Each decoder consists of three major components: a causally masked self-attention mechanism, a cross-attention mechanism, and a feed-forward neural network. The decoder functions in a similar fashion to the encoder, but an additional attention mechanism is inserted which instead draws relevant information from the encodings generated by the encoders. This mechanism can also be called the *encoder–decoder attention*.[1] Like the first encoder, the first decoder takes positional information and embeddings of the output sequence as its input, rather than encodings. The transformer must not use the current or future output to predict an output, so the output sequence must be partially masked to prevent this reverse information flow.[1] This allows for autoregressive text generation. For decoding, all-to-all attention is inappropriate, because a token cannot attend to tokens not yet generated. Thus, the self-attention module in the decoder is causally masked.



One decoder layer

In contrast, the cross-attention mechanism attends to the output vectors of the encoder, which is computed before the decoder starts decoding. Consequently, there is no need for masking in the cross-attention mechanism.

Schematically, we have:

$$\begin{aligned} H' &= \mathbf{MaskedMultiheadAttention}(H, H, H) \\ \mathbf{DecoderLayer}(H) &= \mathbf{FFN}(\mathbf{MultiheadAttention}(H', H^E, H^E)) \end{aligned}$$

where H^E is the matrix with rows being the output vectors from the encoder.

The last decoder is followed by a final un-embedding layer to produce the output probabilities over the vocabulary. Then, one of the tokens is sampled according to the probability, and the decoder can be run again to produce the next token, etc., autoregressively generating output text.

Full transformer architecture

Sublayers

Each encoder layer contains 2 sublayers: the self-attention and the feedforward network. Each decoder layer contains 3 sublayers: the causally masked self-attention, the cross-attention, and the feedforward network.



(a) One encoder layer and one decoder layer. (b) Two encoder layers and two decoder layers. The sublayers are labelled as well.

The final points of detail are the residual connections and layer normalization, (denoted as "LayerNorm", or "LN" in the following), which while conceptually unnecessary, are necessary for numerical stability and convergence.

The residual connections are introduced to avoid vanishing gradient issues and stabilize the training process. They can be expressed by $\boldsymbol{x} \mapsto \boldsymbol{F}(\boldsymbol{x}) + \boldsymbol{x}$, where $\boldsymbol{F}$ is a given component of the transformer. Adding the input $\boldsymbol{x}$ can preserve the input information and avoid issues when the gradient of $\boldsymbol{F}(\boldsymbol{x})$ is close to zero.

Similarly to how the feedforward network modules are applied individually to each vector, the LayerNorm is also applied individually to each vector.

There are two common conventions in use: the *post-LN* and the *pre-LN* convention. In the post-LN convention, the output of each sublayer is

$$\mathbf{LayerNorm}(x + \mathbf{Sublayer}(x))$$

where $\mathbf{Sublayer}(x)$ is the function implemented by the sublayer itself.

In the pre-LN convention, the output of each sublayer is

$$x + \text{Sublayer}(\text{LayerNorm}(x))$$

The original 2017 transformer used the post-LN convention. It was difficult to train and required careful hyperparameter tuning and a "warm-up" in learning rate, where it starts small and gradually increases. The pre-LN convention, proposed several times in 2018,[53] was found to be easier to train, requiring no warm-up, leading to faster convergence.[42]

Pseudocode

The following is the pseudocode for a standard pre-LN encoder–decoder transformer, adapted from *Formal Algorithms for Transformers*.[54]

```
input: Encoder input t_e
       Decoder input t_d
output: Array of probability distributions, with shape
(decoder vocabulary size x length(decoder output
sequence))

/* encoder */
z_e ← encoder.tokenizer(t_e)

for each t in 1:length(z_e) do
    z_e[t] ← encoder.embedding(z_e[t]) +
encoder.positional_embedding(t)

for each l in 1:length(encoder.layers) do
    layer ← encoder.layers[l]

    /* first sublayer */
    z_e_copy ← copy(z_e)
    for each t in 1:length(z_e) do
        z_e[t] ← layer.layer_norm(z_e[t])
    z_e ← layer.multihead_attention(z_e, z_e, z_e)
    for each t in 1:length(z_e) do
        z_e[t] ← z_e[t] + z_e_copy[t]

    /* second sublayer */
    z_e_copy ← copy(z_e)
    for each t in 1:length(z_e) do
        z_e[t] ← layer.layer_norm(z_e[t])
    z_e ← layer.feedforward(z_e)
    for each t in 1:length(z_e) do
        z_e[t] ← z_e[t] + z_e_copy[t]

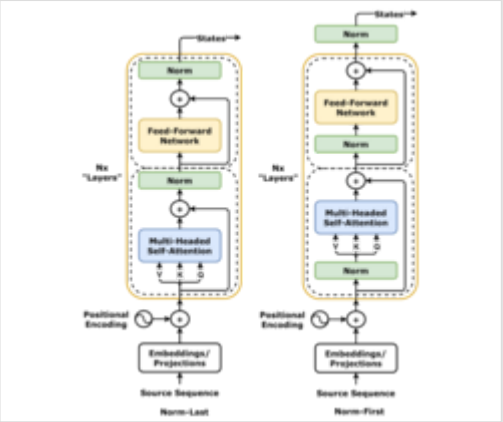
for each t in 1:length(z_e) do
    z_e[t] ← encoder.final_layer_norm(z_e[t])

/* decoder */
z_d ← decoder.tokenizer(t_d)

for each t in 1:length(z_d) do
    z_d[t] ← decoder.embedding(z_d[t]) +
decoder.positional_embedding(t)

for each l in 1:length(decoder.layers) do
        layer ← decoder.layers[l]

        /* first sublayer */
        z_d_copy ← copy(z_d)
        for each t in 1:length(z_d) do
            z_d[t] ← layer.layer_norm(z_d[t])
        z_d ← layer.masked_multihead_attention(z_d, z_d,
z_d)
```



Transformer encoder with norm-first and norm-last



Transformer decoder with norm-first and norm-last



Block diagram for the full transformer architecture

```
        for each t in 1:length(z_d) do
            z_d[t] ← z_d[t] + z_d_copy[t]

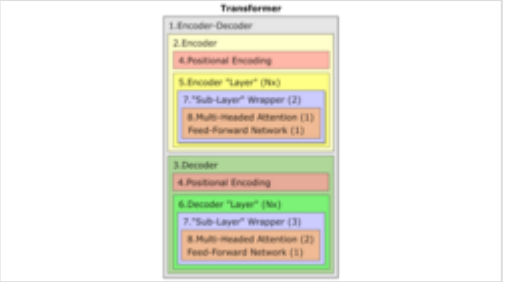
        /* second sublayer */
        z_d_copy ← copy(z_d)
        for each t in 1:length(z_d) do
            z_d[t] ← layer.layer_norm(z_d[t])
        z_d ← layer.multihead_attention(z_d, z_e, z_e)
       for each t in 1:length(z_d) do
           z_d[t] ← z_d[t] + z_d_copy[t]

        /* third sublayer */
        z_d_copy ← copy(z_d)
        for each t in 1:length(z_d) do
            z_d[t] ← layer.layer_norm(z_d[t])
        z_d ← layer.feedforward(z_d)
        for each t in 1:length(z_d) do
            z_d[t] ← z_d[t] + z_d_copy[t]

z_d ← decoder.final_layer_norm(z_d)

output_distributions ← []
for each t in 1:length(z_d) do
    output_distributions.append(decoder.unembed(z_d[t]))

return output_distributions
```



Schematic object hierarchy for the full transformer architecture, in object-oriented programming style

Terminology

The transformer architecture, being modular, allows variations. Several common variations are described here.[45]

An "encoder-only" transformer applies the encoder to map an input text into a sequence of vectors that represent the input text. This is usually used for text embedding and representation learning for downstream applications. BERT is encoder-only. They are less often used currently, as they were found to be not significantly better than training an encoder–decoder transformer, then taking just the encoder.[49] They are also referred to as "all-to-all" or "BERT-like".

A "decoder-only" transformer is not literally decoder-only, since without an encoder, the cross-attention mechanism has nothing to attend to. Thus, the decoder layers in a decoder-only transformer are composed of just two sublayers: the causally masked self-attention, and the feedforward network. This is usually used for text generation and instruction following. The models in the GPT series and Chinchilla series are decoder-only. They are also referred to as "autoregressive" or "causal".

An "encoder–decoder" transformer is generally the same as the original transformer, with 2 sublayers per encoder layer and 3 sublayers per decoder layer, etc. They might have minor architectural improvements, such as alternative activation functions, changing the location of normalization, etc. This is also usually used for text generation and instruction following. The models in the T5 series are encoder–decoder.[45]

A "prefixLM" (prefix language model) is a decoder-only architecture, but with prefix masking, which is different from causal masking. Specifically, it has mask of the form[45]:Figure 3

$$M_{\text{prefixLM}} = \begin{bmatrix} \mathbf{0} & -\infty \\ \mathbf{0} & M_{\text{causal}} \end{bmatrix}$$

where the first columns correspond to the "prefix", and the subsequent columns correspond to the autoregressively generated text based on the prefix. They resemble encoder–decoder models, but has less "sparsity". Such models are rarely used, though they are cited as theoretical possibilities and benchmarked comparisons.[49]

There are also mixed seq2seq models. For example, in 2020, Google Translate replaced the previous RNN-encoder–RNN-decoder model with a transformer-encoder–RNN-decoder model, as transformer-based decoders did not appear to significantly increase quality unlike the encoder, while the RNN decoder was much faster.[36]

Subsequent work

Many improvements have been added to the Transformer architecture. A standard improved configuration is Transformer++, which uses RoPE, RMSNorm, and SwiGLU feedforward module.[55]

Alternative feedforward module

The original transformer's feedforward module is a standard linear-ReLU-linear network. Other feedforward modules were developed. The Llama series and PaLM used SwiGLU;[56] both GPT-1 and BERT[34] used GELU.[57]

Alternative activation functions are often used in combination with Gated Linear Units in the feedforward module.[56]

Alternative normalizations

The normalization used in the transformer can be different from LayerNorm. One example is RMSNorm[58] which is used in the Llama series. Other examples include ScaleNorm[59] and FixNorm.[59]

Alternative positional encodings

Transformers may use other positional encoding methods than sinusoidal.[60]

The original transformer paper reported using a learned positional encoding,[61] but finding it not superior to the sinusoidal one.[1] Later,[62] found that causal masking itself provides enough signal to a transformer decoder that it can learn to implicitly perform absolute positional encoding without the positional encoding module.

Efficient implementation

The transformer model has been implemented in standard deep learning frameworks such as TensorFlow and PyTorch. *Transformers* is a library produced by Hugging Face that supplies transformer-based architectures and pretrained models.[9]

KV caching

When an autoregressive transformer is used for inference, such as generating text, the query vector is different at each step, but the already-computed key and value vectors are always the same. The **KV caching** method saves the computed key and value vectors at each attention block, so that they are not recomputed at each new token. PagedAttention applies memory paging to KV caching.[63][64][65]

If a transformer is used with a baked-in prompt, such as ["You are a customer support agent..."], then the key and value vectors can be computed for the prompt, and saved on disk. The saving in compute is significant when the model is used for many short real-time interactions, such as in online chatbots.

In general, when a user uses an autoregressive transformer to generate a continuation to a sequence of tokens, the model would first perform a forward-pass on this sequence, whereby the KV caches over this sequence are computed. This is called **prefilling**. After prefilling, the actual token generation is called **decoding**. Hyperscalers serving large Transformer models may use **disaggregated inference**, wherein prefilling and decoding are performed on separately specialized hardware.[66]

Continuous Batching

When using autoregressive transformer models, requests can have different input lengths and can finish generating at different times. Continuous batching adds new requests to batch and removes completed requests between decoding iterations instead of waiting for every request in a batch to finish. This improves hardware utilization and can reduce latency and increase throughput for systems that serve many concurrent requests.[67]

Quantization

Transformer models are often quantized for inference, meaning that weights, activations, or both are represented with lower-precision numerical formats than the floating point formats used during training. Quantization can reduce memory use and data transfer costs and can improve inference speed on hardware that supports low precision arithmetic. The trade off is that aggressive quantization can come at the cost of model accuracy or may need calibration and specialized quantization methods to perform well.[68]

FlashAttention

FlashAttention[69] is an algorithm that implements the transformer attention mechanism efficiently on a GPU. It is a communication-avoiding algorithm that performs matrix multiplications in blocks, such that each block fits within the cache of a GPU, and by careful management of the blocks it minimizes data copying between GPU caches (as data movement is slow).

The FlashAttention method is a communication-avoiding algorithm that fuses these operations into a single loop, increasing the arithmetic intensity. It is an online algorithm that computes the following quantities:[70][69]

$$
\begin{aligned}
z_i &= q^T k_i \\
m_i &= \max(z_1, \ldots, z_i) && = \max(m_{i-1}, z_i) \\
\ell_i &= e^{z_1 - m_i} + \cdots + e^{z_i - m_i} && = e^{m_{i-1} - m_i} \ell_{i-1} + e^{z_i - m_i} \\
o_i &= e^{z_1 - m_i} v_1 + \cdots + e^{z_i - m_i} v_i && = e^{m_{i-1} - m_i} o_{i-1} + e^{z_i - m_i} v_i
\end{aligned}
$$

and returns o_N/ℓ_N. In practice, FlashAttention operates over multiple queries and keys per loop iteration, in a similar way as blocked matrix multiplication. If backpropagation is needed, then the output vectors and the intermediate arrays $[m_1, \ldots, m_N], [\ell_1, \ldots, \ell_N]$ are cached, and during the backward pass, attention matrices are rematerialized from these, making it a form of gradient checkpointing.

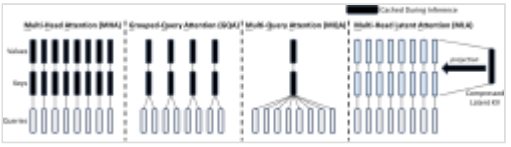
An improved version, FlashAttention-2, was developed to handle longer context lengths.[71]

Multi-Query Attention

Multi-Query Attention (MQA) changes the Multihead Attention mechanism.[72] Whereas normally,

$$\text{MultiheadAttention}(Q, K, V) = \text{Concat}_{i \in [n_{\text{heads}}]} \left(\text{A}$$

with Multi-Query Attention, there is just one W^K, W^V, thus:



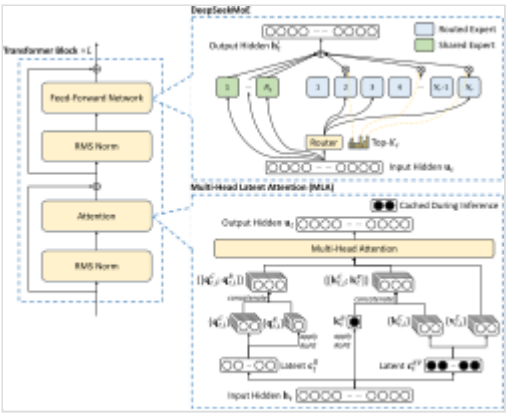
Comparison between several different forms of attention mechanism and the amount of KV caching necessary for each

$$\text{MultiQueryAttention}(Q, K, V) = \text{Concat}_{i \in [n_{\text{heads}}]} \left(\text{Attention}(XW_i^Q, XW^K, XW^V) \right) W^O$$

This has a neutral effect on model quality and training speed, but increases inference speed.

More generally, grouped-query attention (GQA) partitions attention heads into groups, each of which shares the key-value pair. MQA is GQA with one group, while standard Multihead Attention is GQA with the maximal number of groups.[73]

Multihead Latent Attention (MLA) is a low-rank approximation to MQA. Specifically, each hidden vector, before entering the attention mechanism, is first projected to two low-dimensional spaces ("latent space"), one for query and one for key-value (KV vector). This design minimizes the KV cache, as only the low-dimensional KV vector needs to be cached.[74]



The architecture of V2, showing both MLA and a variant of mixture of experts[74]: Figure 2

Speculative decoding

Speculative decoding[75][76] is a method to accelerate token decoding. Similarly to speculative execution in CPUs, future tokens are computed quickly, then verified. If the quickly computed tokens are incorrect, they are discarded and computed slowly.

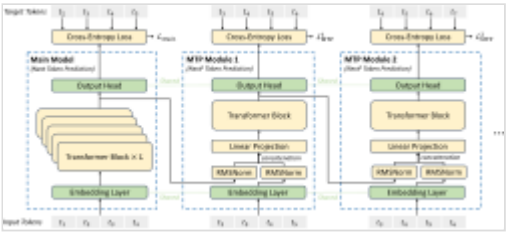
The key factor in speculative decoding is that a transformer decoder can verify faster than it can decode, in the following sense.

Suppose we have two transformer models like GPT-3 and GPT-3-small, both with a context window size of 512. To generate an entire context window autoregressively with greedy decoding with GPT-3, it must be run for 512 times, each time generating a token $x_1, x_2, \ldots, x_{512}$, taking time $512T_{\text{GPT-3}}$. However, if we had some educated guess for the values of these tokens, we could verify all of them in parallel, in one run of the model, by checking that each x_t is indeed the token with the largest log-likelihood in the t-th output.

In speculative decoding, a smaller model or some other simple heuristic is used to generate a few speculative tokens that are subsequently verified by the larger model. For example, suppose we use GPT-3-small to generate four speculative tokens: $\tilde{x}_1, \tilde{x}_2, \tilde{x}_3, \tilde{x}_4$. This only takes $4T_{\text{GPT-3-small}}$. These tokens are then run through the larger GPT-3 in one go. Suppose that $\tilde{x}_1$ and $\tilde{x}_2$ are verified by GPT-3 as what it would have picked, then those are kept, but $\tilde{x}_3$ is not, so $\tilde{x}_3, \tilde{x}_4$ are discarded, and GPT-3 is run on those. This would take $4T_{\text{GPT-3-small}} + 3T_{\text{GPT-3}}$, which might be shorter than $4T_{\text{GPT-3}}$.

For non-greedy decoding, similar ideas apply, except the speculative tokens are accepted or rejected stochastically, in a way that guarantees the final output distribution is the same as if speculative decoding was not used.[75][77]

In Multi-Token Prediction, a single forward pass creates a final embedding vector, which then is un-embedded into a token probability. However, that vector can then be further processed by another transformer block to predict the *next* token, and so on for arbitrarily many steps into the future. This trades off accuracy for speed, since each new token costs just one more transformer block, rather than the entire stack.[78][79]


Multi-token prediction

Sub-quadratic transformers

Training transformer-based architectures can be expensive, especially for long inputs.[80] Many methods have been developed to attempt to address the issue. In the image domain, Swin transformer is an efficient architecture that performs attention inside shifting windows.[81] In the audio domain, SepTr decouples the attention in time and frequency domains.[82] *Long Range Arena* (2020)[83] is a standard benchmark for comparing the behavior of transformer architectures over long inputs.

Alternative attention graphs

The standard attention graph is either all-to-all or causal, both of which scales as $O(N^2)$ where N is the number of tokens in a sequence.

Reformer (2020)[80][84] reduces the computational load from $O(N^2)$ to $O(N \ln N)$ by using locality-sensitive hashing and reversible layers.[85]

Sparse attention[86] uses attention graphs that grows slower than $O(N^2)$. For example, BigBird (2020)[87] uses random small-world networks which grows as $O(N)$.

Ordinary transformers require a memory size that is quadratic in the size of the context window. Attention-free transformers[88] reduce this to a linear dependence while still retaining the advantages of a transformer by linking the key to the value.

Random Feature Attention

Random Feature Attention (2021)[89] uses Fourier random features:

$$\varphi(x) = \frac{1}{\sqrt{D}}[\cos\langle w_1, x\rangle, \sin\langle w_1, x\rangle, \cdots \cos\langle w_D, x\rangle, \sin\langle w_D, x\rangle]^T$$

where $w_1, \dots, w_D$ are independent samples from the normal distribution $N(0, \sigma^2 I)$. This choice of parameters satisfies $\mathbb{E}[\langle \varphi(x), \varphi(y)\rangle] = e^{-\frac{\|x-y\|^2}{2\sigma^2}}$, or

$$e^{\langle x,y\rangle/\sigma^2} = \mathbb{E}[\langle e^{\|x\|^2/2\sigma^2}\varphi(x), e^{\|y\|^2/2\sigma^2}\varphi(y)\rangle] \approx \langle e^{\|x\|^2/2\sigma^2}\varphi(x), e^{\|y\|^2/2\sigma^2}\varphi(y)\rangle$$

Consequently, the one-headed attention, with one query, can be written as

$$\text{Attention}(q, K, V) = \text{softmax}\left(\frac{qK^\text{T}}{\sqrt{d_k}}\right)V \approx \frac{\varphi(q)^T \sum_i e^{\|k_i\|^2/2\sigma^2}\varphi(k_i)v_i^T}{\varphi(q)^T \sum_i e^{\|k_i\|^2/2\sigma^2}\varphi(k_i)}$$

where $\sigma = d_K^{1/4}$. Similarly for multiple queries, and for multihead attention.

This approximation can be computed in linear time, as we can compute the matrix $\varphi(k_i)v_i^T$ first, then multiply it with the query. In essence, we have managed to obtain a more precise version of

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\text{T}}{\sqrt{d_k}}\right)V \approx Q(K^TV/\sqrt{d_k})$$

Performer (2020)[90] uses the same Random Feature Attention, but $w_1, \dots, w_D$ are first independently sampled from the normal distribution $N(0, \sigma^2 I)$, then they are Gram–Schmidt processed.

Multimodality

Transformers can also be used/adapted for modalities (input or output) beyond just text, usually by finding a way to "tokenize" the modality.

Multimodal models can either be trained from scratch, or by finetuning. A 2022 study found that transformers pretrained only on natural language can be finetuned on only 0.03% of parameters and become competitive with LSTMs on a variety of logical and visual tasks, demonstrating transfer learning.[91] The LLaVA was a vision-language model composed of a language model (Vicuna-13B)[92] and a vision model (ViT-L/14), connected by a linear layer. Only the linear layer is finetuned.[93]

Vision transformers adapt the transformer to computer vision by breaking down input images as a series of patches, turning them into vectors, and treating them like embedding vectors of tokens in a standard transformer.

Conformer and later Whisper[94] follow the same pattern for speech recognition, first turning the speech signal into a spectrogram, which is then treated like an image, i.e. broken down into a series of patches, turned into vectors and treated like embedding vector of tokens in a standard transformer.

Perceivers[95][96] are a variant of transformers designed for multimodality.

For image generation, notable architectures are DALL-E 1 (2021), Parti (2022),[97] Phenaki (2023),[98] and Muse (2023).[99] Unlike later models, DALL-E is not a diffusion model. Instead, it uses a decoder-only transformer that autoregressively generates a text, followed by the token representation of an image, which is then converted by a variational autoencoder to an image.[100] Parti is an encoder–decoder transformer, where the encoder processes a text prompt, and the decoder generates a token representation of an image.[101] Muse is an encoder-only transformer that is trained to predict masked image tokens from unmasked image tokens. During generation, all input tokens are masked, and the highest-confidence predictions are included for the next iteration, until all tokens are predicted.[99] Phenaki is a text-to-video model. It is a bidirectional masked transformer conditioned on pre-computed text tokens. The generated tokens are then decoded to a video.[98]

Applications

The transformer has had great success in natural language processing (NLP). Many large language models such as GPT-2, GPT-3, GPT-4, Gemini, AlbertAGPT, Claude, BERT, Grok, XLNet, RoBERTa and ChatGPT demonstrate the ability of transformers to perform a wide variety of NLP-related subtasks and their related real-world applications, including:

- machine translation
- time series prediction
- document summarization
- document generation
- named entity recognition (NER)[102]
- writing computer code based on requirements expressed in natural language.
- speech-to-text

Beyond traditional NLP, the transformer architecture has had success in other applications, such as:

- biological sequence analysis
- video understanding
- protein folding (such as AlphaFold)
- evaluating chess board positions. Using static evaluation alone (that is, with no Minimax search) transformer achieved an Elo of 2895, putting it at grandmaster level.[8]
- service function chain embedding[103]

See also

- seq2seq – Family of machine learning approaches
- Circuit (neural network) – Interpretable computational sub-graphs within artificial neural networks
- Perceiver – Variant of Transformer designed for multimodal data
- Vision transformer – Machine learning model for vision processing
- Large language model – Type of machine learning model
- BERT (language model) – Series of language models developed by Google AI
- Generative pre-trained transformer – Type of large language model
- T5 (language model) – Series of large language models developed by Google AI

Notes

1. Gated recurrent units (2014) further reduced its complexity.
2. Some architectures, such as RWKV (Receptance Weighted Key Value) or state space models, avoid the issue.

References

1. Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N; Kaiser, Łukasz; Polosukhin, Illia (2017). "Attention is All You Need" (https://web.archive.org/web/20240221141113/https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf) (PDF). *31st Conference on Neural Information Processing Systems (NIPS 2017), 4-9 December 2017, Long Beach, CA, USA*. Vol. 30. Curran Associates, Inc. arXiv:1706.03762 (https://arxiv.org/abs/1706.03762). ISBN 978-1-5108-6096-4. Archived from the original (https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf) (PDF) on 2024-02-21. Retrieved 2023-10-31.
2. Hochreiter, Sepp; Schmidhuber, Jürgen (November 1997). "Long Short-Term Memory". *Neural Computation*. **9** (8): 1735–1780. doi:10.1162/neco.1997.9.8.1735 (https://doi.org/10.1162%2Fneco.1997.9.8.1735). PMID 9377276 (https://pubmed.ncbi.nlm.nih.gov/9377276).
3. Bahdanau; Cho, Kyunghyun; Bengio, Yoshua (September 1, 2014). "Neural Machine Translation by Jointly Learning to Align and Translate". arXiv:1409.0473 (https://arxiv.org/abs/1409.0473) [cs.CL (https://arxiv.org/archive/cs.CL)].
4. Luong, Minh-Thang; Pham, Hieu; Manning, Christopher D. (August 17, 2015). "Effective Approaches to Attention-based Neural Machine Translation". arXiv:1508.04025 (https://arxiv.org/abs/1508.04025) [cs.CL (https://arxiv.org/archive/cs.CL)].
5. Parisotto, Emilio; Song, Francis; Rae, Jack; Pascanu, Razvan; Gulcehre, Caglar; Jayakumar, Siddhant; Jaderberg, Max; Kaufman, Raphaël Lopez; Clark, Aidan; Noury, Seb; Botvinick, Matthew; Heess, Nicolas; Hadsell, Raia (2020-11-21). "Stabilizing Transformers for Reinforcement Learning" (https://proceedings.mlr.press/v119/parisotto20a.html). *Proceedings of the 37th International Conference on Machine Learning*. PMLR: 7487–7498. Archived (https://web.archive.org/web/20240809053144/https://proceedings.mlr.press/v119/parisotto20a.html) from the original on 2024-08-09. Retrieved 2024-08-09.

6. Radford, Alec; Jong Wook Kim; Xu, Tao; Brockman, Greg; McLeavey, Christine; Sutskever, Ilya (2022). "Robust Speech Recognition via Large-Scale Weak Supervision". arXiv:2212.04356 (https://arxiv.org/abs/2212.04356) [eess.AS (https://arxiv.org/archive/eess.AS)].
7. Monastirsky, Maxim; Azulay, Osher; Sintov, Avishai (February 2023). "Learning to Throw With a Handful of Samples Using Decision Transformers". *IEEE Robotics and Automation Letters*. **8** (2): 576–583. Bibcode:2023IRAL....8..576M (https://ui.adsabs.harvard.edu/abs/2023IRAL....8..576M). doi:10.1109/LRA.2022.3229266 (https://doi.org/10.1109%2FLRA.2022.3229266).
8. Ruoss, Anian; Delétang, Grégoire; Medapati, Sourabh; Grau-Moya, Jordi; Wenliang, Li; Catt, Elliot; Reid, John; Genewein, Tim (2024-02-07). "Grandmaster-Level Chess Without Search". arXiv:2402.04494v1 (https://arxiv.org/abs/2402.04494v1) [cs.LG (https://arxiv.org/archive/cs.LG)].
9. Wolf, Thomas; Debut, Lysandre; Sanh, Victor; Chaumond, Julien; Delangue, Clement; Moi, Anthony; Cistac, Pierric; Rault, Tim; Louf, Remi; Funtowicz, Morgan; Davison, Joe; Shleifer, Sam; von Platen, Patrick; Ma, Clara; Jernite, Yacine; Plu, Julien; Xu, Canwen; Le Scao, Teven; Gugger, Sylvain; Drame, Mariama; Lhoest, Quentin; Rush, Alexander (2020). "Transformers: State-of-the-Art Natural Language Processing". *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. pp. 38–45. doi:10.18653/v1/2020.emnlp-demos.6 (https://doi.org/10.18653%2Fv1%2F2020.emnlp-demos.6).
10. Feldman, J; Ballard, D (September 1982). "Connectionist models and their properties". *Cognitive Science*. **6** (3): 205–254. doi:10.1016/S0364-0213(82)80001-3 (https://doi.org/10.1016%2FS0364-0213%2882%2980001-3).
11. Rumelhart, David E.; Hinton, Geoffrey E.; McClelland, James L. (1986). "Interactive Processes in Speech Perception". *Parallel Distributed Processing*. doi:10.7551/mitpress/5236.003.0005 (https://doi.org/10.7551%2Fmitpress%2F5236.003.0005). ISBN 978-0-262-29140-8.
12. Giles, C. Lee; Maxwell, Tom (December 1987). "Learning, invariance, and generalization in high-order neural networks". *Applied Optics*. **26** (23): 4972–4978. doi:10.1364/AO.26.004972 (https://doi.org/10.1364%2FAO.26.004972). PMID 20523475 (https://pubmed.ncbi.nlm.nih.gov/20523475).
13. von der Malsburg, Christoph (1981). The Correlation Theory of Brain Function (https://web-archive.southampton.ac.uk/cogprints.org/1380/) (Report). MPI. See Reprint: von Der Malsburg, Christoph (1994). "The Correlation Theory of Brain Function". *Models of Neural Networks*. Physics of Neural Networks. pp. 95–119. doi:10.1007/978-1-4612-4320-5_2 (https://doi.org/10.1007%2F978-1-4612-4320-5_2). ISBN 978-1-4612-8736-0.
14. Feldman, Jerome A. (December 1982). "Dynamic connections in neural networks". *Biological Cybernetics*. **46** (1): 27–39. doi:10.1007/BF00335349 (https://doi.org/10.1007%2FBF00335349). PMID 6307398 (https://pubmed.ncbi.nlm.nih.gov/6307398).
15. Hinton, Geoffrey E.; Plaut, David C. (1987). "Using Fast Weights to Deblur Old Memories" (https://escholarship.org/uc/item/0570j1dp). *Program of the Ninth Annual Conference of the Cognitive Science Society: 16-18 July 1987, Seattle, Washington*. Psychology Press. pp. 177–186. ISBN 978-0-8058-0166-8.
16. Schmidhuber, Jürgen (January 1992). "Learning to Control Fast-Weight Memories: An Alternative to Dynamic Recurrent Networks". *Neural Computation*. **4** (1): 131–139. doi:10.1162/neco.1992.4.1.131 (https://doi.org/10.1162%2Fneco.1992.4.1.131).
17. Katharopoulos, Angelos; Vyas, Apoorv; Pappas, Nikolaos; Fleuret, François (2020). "Transformers are RNNs: Fast autoregressive Transformers with linear attention" (https://infoscience.epfl.ch/handle/20.500.14299/181706). *ICML 2020*. PMLR. pp. 5156–5165.

18. Schlag, Imanol; Irie, Kazuki; Schmidhuber, Jürgen (2021). "Linear Transformers Are Secretly Fast Weight Programmers" (https://icml.cc/virtual/2021/spotlight/10588). *ICML 2021*. Springer. pp. 9355–9366. Archived (https://web.archive.org/web/20260520061714/https://icml.cc/virtual/2021/spotlight/10588) from the original on 2026-05-20. Retrieved 2026-05-20.
19. Cho, Kyunghyun; van Merriënboer, Bart; Gulcehre, Caglar; Bahdanau, Dzmitry; Bougares, Fethi; Schwenk, Holger; Bengio, Yoshua (October 2014). "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation" (https://aclanthology.org/D14-1179). In Moschitti, Alessandro; Pang, Bo; Daelemans, Walter (eds.). *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Doha, Qatar: Association for Computational Linguistics. pp. 1724–1734. arXiv:1406.1078 (https://arxiv.org/abs/1406.1078). doi:10.3115/v1/D14-1179 (https://doi.org/10.3115%2Fv1%2FD14-1179). Archived (https://web.archive.org/web/20240906214240/https://aclanthology.org/D14-1179/) from the original on 2024-09-06. Retrieved 2024-08-09.
20. Sutskever, Ilya; Vinyals, Oriol; Le, Quoc Viet (14 Dec 2014). "Sequence to sequence learning with neural networks". arXiv:1409.3215 (https://arxiv.org/abs/1409.3215) [cs.CL (https://arxiv.org/archive/cs.CL)]. [first version posted to arXiv on 10 Sep 2014]
21. Chung, Junyoung; Gulcehre, Caglar; Cho, KyungHyun; Bengio, Yoshua (2014). "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling". arXiv:1412.3555 (https://arxiv.org/abs/1412.3555) [cs.NE (https://arxiv.org/archive/cs.NE)].
22. Gruber, Nicole; Jockisch, Alfred (30 June 2020). "Are GRU Cells More Specific and LSTM Cells More Sensitive in Motive Classification of Text?" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7861254). *Frontiers in Artificial Intelligence*. **3** 40. doi:10.3389/frai.2020.00040 (https://doi.org/10.3389%2Ffrai.2020.00040). PMC 7861254 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7861254). PMID 33733157 (https://pubmed.ncbi.nlm.nih.gov/33733157).
23. Sutskever, Ilya; Vinyals, Oriol; Le, Quoc V (2014). "Sequence to Sequence Learning with Neural Networks" (https://web.archive.org/web/20250127094117/https://proceedings.neurips.cc/paper/2014/hash/a14ac55a4f27472c5d894ec1c3c743d2-Abstract.html). *Advances in Neural Information Processing Systems*. **27**. Curran Associates, Inc. arXiv:1409.3215 (https://arxiv.org/abs/1409.3215). Archived from the original (https://proceedings.neurips.cc/paper/2014/hash/a14ac55a4f27472c5d894ec1c3c743d2-Abstract.html) on 2025-01-27. Retrieved 2024-07-23.
24. Luong, Minh-Thang; Pham, Hieu; Manning, Christopher D. (2015). "Effective Approaches to Attention-based Neural Machine Translation". arXiv:1508.04025 (https://arxiv.org/abs/1508.04025) [cs.CL (https://arxiv.org/archive/cs.CL)].
25. Wu, Yonghui; et al. (2016-09-01). "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation". arXiv:1609.08144 (https://arxiv.org/abs/1609.08144) [cs.CL (https://arxiv.org/archive/cs.CL)].
26. Lewis-Kraus, Gideon (2016-12-14). "The Great A.I. Awakening" (https://web.archive.org/web/20230524052626/https://www.nytimes.com/2016/12/14/magazine/the-great-ai-awakening.html). *The New York Times*. Archived from the original (https://www.nytimes.com/2016/12/14/magazine/the-great-ai-awakening.html) on 24 May 2023. Retrieved 2023-06-22.
27. Parikh, Ankur P.; Täckström, Oscar; Das, Dipanjan; Uszkoreit, Jakob (2016-09-25). "A Decomposable Attention Model for Natural Language Inference". arXiv:1606.01933 (https://arxiv.org/abs/1606.01933) [cs.CL (https://arxiv.org/archive/cs.CL)].
28. Levy, Steven. "8 Google Employees Invented Modern AI. Here's the Inside Story" (https://www.wired.com/story/eight-google-employees-invented-modern-ai-transformers-paper/). *Wired*. Archived (https://web.archive.org/web/20240320101528/https://www.wired.com/story/eight-google-employees-invented-modern-ai-transformers-paper/) from the original on 20 Mar 2024. Retrieved 2024-08-06.

29. Cheng, Jianpeng; Dong, Li; Lapata, Mirella (November 2016). "Long Short-Term Memory-Networks for Machine Reading" (https://aclanthology.org/D16-1053/). In Su, Jian; Duh, Kevin; Carreras, Xavier (eds.). *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Austin, Texas: Association for Computational Linguistics. pp. 551–561. doi:10.18653/v1/D16-1053 (https://doi.org/10.18653%2Fv1%2FD16-1053). Archived (https://web.archive.org/web/20240827143929/https://aclanthology.org/D16-1053/) from the original on 2024-08-27. Retrieved 2024-08-27.
30. Peng, Bo; Alcaide, Eric; Anthony, Quentin; Albalak, Alon; Arcadinho, Samuel; Biderman, Stella; Cao, Huanqi; Cheng, Xin; Chung, Michael (2023-12-10). "RWKV: Reinventing RNNs for the transformer Era". arXiv:2305.13048 (https://arxiv.org/abs/2305.13048) [cs.CL (https://arxiv.org/archive/cs.CL)].
31. Marche, Stephen (2024-08-23). "Was Linguistic A.I. Created by Accident?" (https://www.newyorker.com/science/annals-of-artificial-intelligence/was-linguistic-ai-created-by-accident). *The New Yorker*. Retrieved 2024-08-27.
32. Vaswani, Ashish; Bengio, Samy; Brevdo, Eugene; Chollet, Francois; Gomez, Aidan; Gouws, Stephan; Jones, Llion; Kaiser, Łukasz; Kalchbrenner, Nal; Parmar, Niki; Sepassi, Ryan; Shazeer, Noam; Uszkoreit, Jakob (March 2018). Cherry, Colin; Neubig, Graham (eds.). "Tensor2Tensor for Neural Machine Translation" (https://aclanthology.org/W18-1819/). *Proceedings of the 13th Conference of the Association for Machine Translation in the Americas (Volume 1: Research Track)*. Boston, MA: Association for Machine Translation in the Americas: 193–199. Archived (https://web.archive.org/web/20260429230143/https://aclanthology.org/W18-1819/) from the original on 2026-04-29. Retrieved 2026-03-31.
33. Kaiser, Łukasz (2017-06-19). "Accelerating Deep Learning Research with the Tensor2Tensor Library" (https://research.google/blog/accelerating-deep-learning-research-with-the-tensor2tensor-library/). *Google Research Blog*. Archived (https://web.archive.org/web/20260415200010/https://research.google/blog/accelerating-deep-learning-research-with-the-tensor2tensor-library/) from the original on 2026-04-15. Retrieved 2026-03-31.
34. Devlin, Jacob; Chang, Ming-Wei; Lee, Kenton; Toutanova, Kristina (11 October 2018). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". arXiv:1810.04805v2 (https://arxiv.org/abs/1810.04805v2) [cs.CL (https://arxiv.org/archive/cs.CL)].
35. "Google: BERT now used on almost every English query" (https://searchengineland.com/google-bert-used-on-almost-every-english-query-342193). *Search Engine Land*. 2020-10-15. Archived (https://web.archive.org/web/20220506220519/https://searchengineland.com/google-bert-used-on-almost-every-english-query-342193) from the original on 2022-05-06. Retrieved 2020-11-24.
36. Caswell, Isaac; Liang, Bowen (June 8, 2020). "Recent Advances in Google Translate" (https://research.google/blog/recent-advances-in-google-translate/). *Google Research*. Archived (https://web.archive.org/web/20240704042433/https://research.google/blog/recent-advances-in-google-translate/) from the original on 4 Jul 2024. Retrieved 2024-08-07.
37. "The inside story of how ChatGPT was built from the people who made it" (https://www.technologyreview.com/2023/03/03/1069311/inside-story-oral-history-how-chatgpt-built-openai/). *MIT Technology Review*. Archived (https://web.archive.org/web/20230303093219/https://www.technologyreview.com/2023/03/03/1069311/inside-story-oral-history-how-chatgpt-built-openai/) from the original on 2023-03-03. Retrieved 2024-08-06.
38. "Introducing ChatGPT" (https://openai.com/index/chatgpt/). *OpenAI*. 2022-11-30. Archived (https://web.archive.org/web/20260405004654/https://openai.com/index/chatgpt/) from the original on 2026-04-05. Retrieved 2026-05-16.
39. Deign, Jason (December 6, 2024). "How ChatGPT changed… well, almost everything" (https://newsroom.cisco.com/c/r/newsroom/en/us/a/y2024/m12/how-chatgpt-changed-well-almost-everything.html). *Cisco Newsroom*. Retrieved 2026-10-22.

40. Kaiser, Lukasz; Gomez, Aidan N.; Shazeer, Noam; Vaswani, Ashish; Parmar, Niki; Jones, Llion; Uszkoreit, Jakob (2017-06-16). "One Model To Learn Them All". arXiv:1706.05137v1 (https://arxiv.org/abs/1706.05137v1) [cs.LG (https://arxiv.org/archive/cs.LG)].
41. Liu, Zhuang; Mao, Hanzi; Wu, Chao-Yuan; Feichtenhofer, Christoph; Darrell, Trevor; Xie, Saining (2022). *A ConvNet for the 2020s* (https://openaccess.thecvf.com/content/CVPR2022/html/Liu_A_ConvNet_for_the_2020s_CVPR_2022_paper.html). Conference on Computer Vision and Pattern Recognition (CVPR). pp. 11976–11986.
42. Xiong, Ruibin; Yang, Yunchang; He, Di; Zheng, Kai; Zheng, Shuxin; Xing, Chen; Zhang, Huishuai; Lan, Yanyan; Wang, Liwei; Liu, Tie-Yan (2020-06-29). "On Layer Normalization in the Transformer Architecture". arXiv:2002.04745 (https://arxiv.org/abs/2002.04745) [cs.LG (https://arxiv.org/archive/cs.LG)].
43. Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N; Kaiser, Ł ukasz; Polosukhin, Illia (2017). "Attention is All you Need" (https://papers.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html). *Advances in Neural Information Processing Systems*. **30**. Curran Associates, Inc.
44. Narayanan, Deepak; Harlap, Aaron; Phanishayee, Amar; Seshadri, Vivek; Devanur, Nikhil R.; Ganger, Gregory R.; Gibbons, Phillip B.; Zaharia, Matei (2019). "PipeDream: Generalized pipeline parallelism for DNN training". *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. pp. 1–15. doi:10.1145/3341301.3359646 (https://doi.org/10.1145%2F3341301.3359646). ISBN 978-1-4503-6873-5.
45. Raffel, Colin; Shazeer, Noam; Roberts, Adam; Lee, Katherine; Narang, Sharan; Matena, Michael; Zhou, Yanqi; Li, Wei; Liu, Peter J. (2020). "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (https://www.jmlr.org/papers/v21/20-074.html). *Journal of Machine Learning Research*. **21** (140): 1–67. Archived (https://web.archive.org/web/20260410072046/https://jmlr.org/papers/v21/20-074.html) from the original on 2026-04-10. Retrieved 2026-04-18.
46. Raffel, Colin; Shazeer, Noam; Roberts, Adam; Lee, Katherine; Narang, Sharan; Matena, Michael; Zhou, Yanqi; Li, Wei; Liu, Peter J. (2019). "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". arXiv:1910.10683 (https://arxiv.org/abs/1910.10683) [cs.LG (https://arxiv.org/archive/cs.LG)].
47. "Masked language modeling" (https://huggingface.co/docs/transformers/tasks/masked_language_modeling). *huggingface.co*. Archived (https://web.archive.org/web/20231010083915/https://huggingface.co/docs/transformers/tasks/masked_language_modeling) from the original on 2023-10-10. Retrieved 2023-10-05.
48. "Causal language modeling" (https://huggingface.co/docs/transformers/tasks/language_modeling). *huggingface.co*. Archived (https://web.archive.org/web/20231010083913/https://huggingface.co/docs/transformers/tasks/language_modeling) from the original on 2023-10-10. Retrieved 2023-10-05.
49. Tay, Yi; Dehghani, Mostafa; Tran, Vinh Q.; Garcia, Xavier; Wei, Jason; Wang, Xuezhi; Chung, Hyung Won; Shakeri, Siamak; Bahri, Dara (2023-02-28). "UL2: Unifying Language Learning Paradigms". arXiv:2205.05131 (https://arxiv.org/abs/2205.05131) [cs.CL (https://arxiv.org/archive/cs.CL)].
50. Team, Keras. "Keras documentation: GPT2Backbone model" (https://keras.io/api/keras_nlp/models/gpt2/gpt2_backbone/). *keras.io*. Archived (https://web.archive.org/web/20240808085714/https://keras.io/api/keras_nlp/models/gpt2/gpt2_backbone/) from the original on 2024-08-08. Retrieved 2024-08-08.

51. Clark, Kevin; Khandelwal, Urvashi; Levy, Omer; Manning, Christopher D. (August 2019). "What Does BERT Look at? An Analysis of BERT's Attention" (https://www.aclweb.org/anthology/W19-4828). *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. Florence, Italy: Association for Computational Linguistics: 276–286. arXiv:1906.04341 (https://arxiv.org/abs/1906.04341). doi:10.18653/v1/W19-4828 (https://doi.org/10.18653%2Fv1%2FW19-4828). Archived (https://web.archive.org/web/20201021211357/https://www.aclweb.org/anthology/W19-4828/) from the original on 2020-10-21. Retrieved 2020-05-20.
52. Yang, Zhilin; Dai, Zihang; Yang, Yiming; Carbonell, Jaime; Salakhutdinov, Russ R; Le, Quoc V (2019). "XLNet: Generalized Autoregressive Pretraining for Language Understanding" (https://proceedings.neurips.cc/paper/2019/hash/dc6a7e655d7e5840e66733e9ee67cc69-Abstract.html). *Advances in Neural Information Processing Systems*. **32**. Curran Associates, Inc. arXiv:1906.08237 (https://arxiv.org/abs/1906.08237). Archived (https://web.archive.org/web/20240809053521/https://proceedings.neurips.cc/paper/2019/hash/dc6a7e655d7e5840e66733e9ee67cc69-Abstract.html) from the original on 2024-08-09. Retrieved 2024-08-09.
53. Wang, Qiang; Li, Bei; Xiao, Tong; Zhu, Jingbo; Li, Changliang; Wong, Derek F.; Chao, Lidia S. (2019-06-04). "Learning Deep Transformer Models for Machine Translation". arXiv:1906.01787 (https://arxiv.org/abs/1906.01787) [cs.CL (https://arxiv.org/archive/cs.CL)].
54. Phuong, Mary; Hutter, Marcus (2022-07-19). "Formal Algorithms for Transformers". arXiv:2207.09238 (https://arxiv.org/abs/2207.09238) [cs.LG (https://arxiv.org/archive/cs.LG)].
55. Gu, Albert; Dao, Tri (2023-12-01). "Mamba: Linear-Time Sequence Modeling with Selective State Spaces". arXiv:2312.00752 (https://arxiv.org/abs/2312.00752) [cs.LG (https://arxiv.org/archive/cs.LG)].
56. Shazeer, Noam (2020-02-01). "GLU Variants Improve Transformer". arXiv:2002.05202 (https://arxiv.org/abs/2002.05202) [cs.LG (https://arxiv.org/archive/cs.LG)].
57. Hendrycks, Dan; Gimpel, Kevin (2016-06-27). "Gaussian Error Linear Units (GELUs)". arXiv:1606.08415v5 (https://arxiv.org/abs/1606.08415v5) [cs.LG (https://arxiv.org/archive/cs.LG)].
58. Zhang, Biao; Sennrich, Rico (2019). "Root Mean Square Layer Normalization" (https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html). *Advances in Neural Information Processing Systems*. **32**. Curran Associates, Inc. arXiv:1910.07467 (https://arxiv.org/abs/1910.07467). Archived (https://web.archive.org/web/20240917113737/https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html) from the original on 2024-09-17. Retrieved 2024-08-09.
59. Nguyen, Toan Q.; Salazar, Julian (2019-11-02). Niehues, Jan; Cattoni, Rolando; Stüker, Sebastian; Negri, Matteo; Turchi, Marco; Ha, Thanh-Le; Salesky, Elizabeth; Sanabria, Ramon; Barrault, Loic (eds.). "Transformers without Tears: Improving the Normalization of Self-Attention" (https://aclanthology.org/2019.iwslt-1.17). *Proceedings of the 16th International Conference on Spoken Language Translation*. Hong Kong: Association for Computational Linguistics. arXiv:1910.05895 (https://arxiv.org/abs/1910.05895). doi:10.5281/zenodo.3525484 (https://doi.org/10.5281%2Fzenodo.3525484). Archived (https://web.archive.org/web/20240809185028/https://aclanthology.org/2019.iwslt-1.17/) from the original on 2024-08-09. Retrieved 2024-08-09.
60. Dufter, Philipp; Schmitt, Martin; Schütze, Hinrich (September 2022). "Position Information in Transformers: An Overview" (https://doi.org/10.1162%2Fcoli_a_00445). *Computational Linguistics*. **48** (3): 733–763. arXiv:2102.11090 (https://arxiv.org/abs/2102.11090). doi:10.1162/coli_a_00445 (https://doi.org/10.1162%2Fcoli_a_00445).
61. Gehring, Jonas; Auli, Michael; Grangier, David; Yarats, Denis; Dauphin, Yann N. (2017-07-17). "Convolutional Sequence to Sequence Learning" (https://proceedings.mlr.press/v70/gehring17a.html). *Proceedings of the 34th International Conference on Machine Learning*. PMLR: 1243–1252. Archived (https://web.archive.org/web/20240809184100/https://proceedings.mlr.press/v70/gehring17a.html) from the original on 2024-08-09. Retrieved 2024-08-09.

62. Haviv, Adi; Ram, Ori; Press, Ofir; Izsak, Peter; Levy, Omer (2022-12-05). "Transformer Language Models without Positional Encodings Still Learn Positional Information". arXiv:2203.16634 (https://arxiv.org/abs/2203.16634) [cs.CL (https://arxiv.org/archive/cs.CL)].
63. Kwon, Woosuk; Li, Zhuohan; Zhuang, Siyuan; Sheng, Ying; Zheng, Lianmin; Yu, Cody Hao; Gonzalez, Joseph; Zhang, Hao; Stoica, Ion (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention". *Proceedings of the 29th Symposium on Operating Systems Principles*. pp. 611–626. doi:10.1145/3600006.3613165 (https://doi.org/10.1145%2F3600006.3613165). ISBN 979-8-4007-0229-7.
64. "vllm-project/vllm" (https://github.com/vllm-project/vllm). vLLM. 2024-06-20. Archived (https://web.archive.org/web/20240618204642/https://github.com/vllm-project/vllm) from the original on 2024-06-18. Retrieved 2024-06-20.
65. Zhuohan Li, Woosuk Kwon; Zhuang, Siyuan; Sheng, Ying; Zheng, Lianmin; Yu, Cody; Gonzalez, Joey; Zhang, Hao; Stoica, Ion (2023-06-20). "vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention" (https://blog.vllm.ai/2023/06/20/vllm.html). *vLLM Blog*. Archived (https://web.archive.org/web/20240620205208/https://blog.vllm.ai/2023/06/20/vllm.html) from the original on 2024-06-20. Retrieved 2024-06-20.
66. Hu, Cunchen; Huang, Heyang; Xu, Liangliang; Chen, Xusheng; Xu, Jiang; Chen, Shuang; Feng, Hao; Wang, Chenxi; Wang, Sa (2024-01-20). "Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads". arXiv:2401.11181 (https://arxiv.org/abs/2401.11181) [cs.DC (https://arxiv.org/archive/cs.DC)].
67. Yu, Gyeong-In; Jeong, Joo Seong; Kim, Geon-Woo; Kim, Soojeong; Chun, Byung-Gon (2022). *Orca: A Distributed Serving System for Transformer-Based Generative Models* (https://www.usenix.org/conference/osdi22/presentation/yu). pp. 521–538. ISBN 978-1-939133-28-1.
68. Dettmers, Tim; Lewis, Mike; Belkada, Younes; Zettlemoyer, Luke (2022-08-15). "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". arXiv:2208.07339v2 (https://arxiv.org/abs/2208.07339v2) [cs.LG (https://arxiv.org/archive/cs.LG)].
69. Dao, Tri; Ermon, Stefano; Fu, Dan; Ré, Christopher; Rudra, Atri (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". *Advances in Neural Information Processing Systems 35*. pp. 16344–16359. arXiv:2205.14135 (https://arxiv.org/abs/2205.14135). doi:10.52202/068431-1189 (https://doi.org/10.52202%2F068431-1189). ISBN 978-1-7138-7108-8.
70. Milakov, Maxim; Gimelshein, Natalia (2018). "Online normalizer calculation for softmax". arXiv:1805.02867 (https://arxiv.org/abs/1805.02867) [cs.PF (https://arxiv.org/archive/cs.PF)].
71. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning" (https://crfm.stanford.edu/2023/07/17/flash2.html). *crfm.stanford.edu*. Archived (https://web.archive.org/web/20230718103613/https://crfm.stanford.edu/2023/07/17/flash2.html) from the original on 2023-07-18. Retrieved 2023-07-18.
72. Chowdhery, Aakanksha; Narang, Sharan; Devlin, Jacob; Bosma, Maarten; Mishra, Gaurav; Roberts, Adam; Barham, Paul; Chung, Hyung Won; Sutton, Charles; Gehrmann, Sebastian; Schuh, Parker; Shi, Kensen; Tsvyashchenko, Sasha; Maynez, Joshua; Rao, Abhishek (2022-04-01). "PaLM: Scaling Language Modeling with Pathways". arXiv:2204.02311 (https://arxiv.org/abs/2204.02311) [cs.CL (https://arxiv.org/archive/cs.CL)].
73. Ainslie, Joshua; Lee-Thorp, James; de Jong, Michiel; Zemlyanskiy, Yury; Lebrón, Federico; Sanghai, Sumit (2023-12-23). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints". arXiv:2305.13245 (https://arxiv.org/abs/2305.13245) [cs.CL (https://arxiv.org/archive/cs.CL)].
74. DeepSeek-AI; Liu, Aixin; Feng, Bei; Wang, Bin; Wang, Bingxuan; Liu, Bo; Zhao, Chenggang; Dengr, Chengqi; Ruan, Chong (19 June 2024). "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model". arXiv:2405.04434 (https://arxiv.org/abs/2405.04434) [cs.CL (https://arxiv.org/archive/cs.CL)]..

75. Leviathan, Yaniv; Kalman, Matan; Matias, Yossi (2023-05-18). "Fast Inference from Transformers via Speculative Decoding". arXiv:2211.17192 (https://arxiv.org/abs/2211.17192) [cs.LG (https://arxiv.org/archive/cs.LG)].
76. Fu, Yao (2023-12-11). "Towards 100x Speedup: Full Stack Transformer Inference Optimization" (https://yaofu.notion.site/Towards-100x-Speedup-Full-Stack-Transformer-Inference-Optimization-43124c3688e14cffaf2f1d6cbdf26c6c). *yaofu.notion.site*.
77. Chen, Charlie; Borgeaud, Sebastian; Irving, Geoffrey; Lespiau, Jean-Baptiste; Sifre, Laurent; Jumper, John (2023-02-02). "Accelerating Large Language Model Decoding with Speculative Sampling". arXiv:2302.01318 (https://arxiv.org/abs/2302.01318) [cs.CL (https://arxiv.org/archive/cs.CL)].
78. Gloeckle, Fabian; Badr Youbi Idrissi; Rozière, Baptiste; Lopez-Paz, David; Synnaeve, Gabriel (2024). "Better & Faster Large Language Models via Multi-token Prediction". arXiv:2404.19737 (https://arxiv.org/abs/2404.19737) [cs.CL (https://arxiv.org/archive/cs.CL)].
79. DeepSeek-AI; et al. (2024). "DeepSeek-V3 Technical Report". arXiv:2412.19437 (https://arxiv.org/abs/2412.19437) [cs.CL (https://arxiv.org/archive/cs.CL)].
80. Kitaev, Nikita; Kaiser, Łukasz; Levskaya, Anselm (2020). "Reformer: The Efficient Transformer". arXiv:2001.04451 (https://arxiv.org/abs/2001.04451) [cs.LG (https://arxiv.org/archive/cs.LG)].
81. Liu, Ze; Lin, Yutong; Cao, Yue; Hu, Han; Wei, Yixuan; Zhang, Zheng; Lin, Stephen; Guo, Baining (2021). "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows". *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE. pp. 9992–10002. arXiv:2103.14030 (https://arxiv.org/abs/2103.14030). doi:10.1109/ICCV48922.2021.00986 (https://doi.org/10.1109%2FICCV48922.2021.00986). ISBN 978-1-6654-2812-5.
82. Ristea, Nicolaea Catalin; Ionescu, Radu Tudor; Khan, Fahad Shahbaz (2022-09-18). "SepTr: Separable Transformer for Audio Spectrogram Processing" (https://www.isca-archive.org/interspeech_2022/ristea22_interspeech.html). *Interspeech*. ISCA: 4103–4107. arXiv:2203.09581 (https://arxiv.org/abs/2203.09581). doi:10.21437/Interspeech.2022-249 (https://doi.org/10.21437%2FInterspeech.2022-249).
83. Tay, Yi; Dehghani, Mostafa; Abnar, Samira; Shen, Yikang; Bahri, Dara; Pham, Philip; Rao, Jinfeng; Yang, Liu; Ruder, Sebastian; Metzler, Donald (2020-11-08). "Long Range Arena: A Benchmark for Efficient Transformers". arXiv:2011.04006 (https://arxiv.org/abs/2011.04006) [cs.LG (https://arxiv.org/archive/cs.LG)].
84. "Reformer: The Efficient Transformer" (https://ai.googleblog.com/2020/01/reformer-efficient-transformer.html). *Google AI Blog*. 16 January 2020. Archived (https://web.archive.org/web/20201022210019/https://ai.googleblog.com/2020/01/reformer-efficient-transformer.html) from the original on 2020-10-22. Retrieved 2020-10-22.
85. Gomez, Aidan N; Ren, Mengye; Urtasun, Raquel; Grosse, Roger B (2017). "The Reversible Residual Network: Backpropagation Without Storing Activations" (https://proceedings.neurips.cc/paper/2017/hash/f9be311e65d81a9ad8150a60844bb94c-Abstract.html). *Advances in Neural Information Processing Systems*. **30**. Curran Associates, Inc. arXiv:1707.04585 (https://arxiv.org/abs/1707.04585). Archived (https://web.archive.org/web/20240811200731/https://proceedings.neurips.cc/paper/2017/hash/f9be311e65d81a9ad8150a60844bb94c-Abstract.html) from the original on 2024-08-11. Retrieved 2024-08-11.
86. Child, Rewon; Gray, Scott; Radford, Alec; Sutskever, Ilya (2019-04-23). "Generating Long Sequences with Sparse Transformers". arXiv:1904.10509 (https://arxiv.org/abs/1904.10509) [cs.LG (https://arxiv.org/archive/cs.LG)].
87. "Constructing Transformers For Longer Sequences with Sparse Attention Methods" (https://ai.googleblog.com/2021/03/constructing-transformers-for-longer.html). *Google AI Blog*. 25 March 2021. Archived (https://web.archive.org/web/20210918150757/https://ai.googleblog.com/2021/03/constructing-transformers-for-longer.html) from the original on 2021-09-18. Retrieved 2021-05-28.

88. Zhai, Shuangfei; Talbott, Walter; Srivastava, Nitish; Huang, Chen; Goh, Hanlin; Zhang, Ruixiang; Susskind, Josh (2021-09-21). "An Attention Free Transformer". arXiv:2105.14103 (https://arxiv.org/abs/2105.14103) [cs.LG (https://arxiv.org/archive/cs.LG)].
89. Peng, Hao; Pappas, Nikolaos; Yogatama, Dani; Schwartz, Roy; Smith, Noah A.; Kong, Lingpeng (2021-03-19). "Random Feature Attention". arXiv:2103.02143 (https://arxiv.org/abs/2103.02143) [cs.CL (https://arxiv.org/archive/cs.CL)].
90. Choromanski, Krzysztof; Likhosherstov, Valerii; Dohan, David; Song, Xingyou; Gane, Andreea; Sarlos, Tamas; Hawkins, Peter; Davis, Jared; Mohiuddin, Afroz; Kaiser, Lukasz; Belanger, David; Colwell, Lucy; Weller, Adrian (2020-09-30). "Rethinking Attention with Performers". arXiv:2009.14794 (https://arxiv.org/abs/2009.14794) [cs.LG (https://arxiv.org/archive/cs.LG)].
91. Lu, Kevin; Grover, Aditya; Abbeel, Pieter; Mordatch, Igor (2022). "Frozen Pretrained Transformers as Universal Computation Engines". *Proceedings of the AAAI Conference on Artificial Intelligence*. **36** (7): 7628–7636. doi:10.1609/aaai.v36i7.20729 (https://doi.org/10.1609%2Faaai.v36i7.20729).
92. "Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality | LMSYS Org" (https://lmsys.org/blog/2023-03-30-vicuna). *lmsys.org*. 30 March 2023. Archived (https://web.archive.org/web/20240812055843/https://lmsys.org/blog/2023-03-30-vicuna/) from the original on 2024-08-12. Retrieved 2024-08-11.
93. Liu, Haotian; Li, Chunyuan; Wu, Qingyang; Lee, Yong Jae (2023-12-15). "Visual Instruction Tuning" (https://proceedings.neurips.cc/paper_files/paper/2023/hash/6dcf277ea32ce3288914faf369fe6de0-Abstract-Conference.html). *Advances in Neural Information Processing Systems*. Vol. 36. pp. 34892–34916. doi:10.52202/075280-1516 (https://doi.org/10.52202%2F075280-1516). ISBN 978-1-7138-9911-2. Archived (https://web.archive.org/web/20240926183648/https://proceedings.neurips.cc//paper_files/paper/2023/hash/6dcf277ea32ce3288914faf369fe6de0-Abstract-Conference.html) from the original on 2024-09-26. Retrieved 2024-08-11.
94. Radford, Alec; Kim, Jong Wook; Xu, Tao; Brockman, Greg; McLeavey, Christine; Sutskever, Ilya (2022). "Robust Speech Recognition via Large-Scale Weak Supervision". arXiv:2212.04356 (https://arxiv.org/abs/2212.04356) [eess.AS (https://arxiv.org/archive/eess.AS)].
95. Jaegle, Andrew; Gimeno, Felix; Brock, Andrew; Zisserman, Andrew; Vinyals, Oriol; Carreira, Joao (2021-06-22). "Perceiver: General Perception with Iterative Attention". arXiv:2103.03206 (https://arxiv.org/abs/2103.03206) [cs.CV (https://arxiv.org/archive/cs.CV)].
96. Jaegle, Andrew; Borgeaud, Sebastian; Alayrac, Jean-Baptiste; Doersch, Carl; Ionescu, Catalin; Ding, David; Koppula, Skanda; Zoran, Daniel; Brock, Andrew; Shelhamer, Evan; Hénaff, Olivier (2021-08-02). "Perceiver IO: A General Architecture for Structured Inputs & Outputs". arXiv:2107.14795 (https://arxiv.org/abs/2107.14795) [cs.LG (https://arxiv.org/archive/cs.LG)].
97. "Parti: Pathways Autoregressive Text-to-Image Model" (https://sites.research.google/parti/). *sites.research.google*. Archived (https://web.archive.org/web/20240810094406/https://sites.research.google/parti/) from the original on 2024-08-10. Retrieved 2024-08-09.
98. Villegas, Ruben; Babaeizadeh, Mohammad; Kindermans, Pieter-Jan; Moraldo, Hernan; Zhang, Han; Saffar, Mohammad Taghi; Castro, Santiago; Kunze, Julius; Erhan, Dumitru (2022-09-29). "Phenaki: Variable Length Video Generation from Open Domain Textual Descriptions". arXiv:2210.02399 (https://arxiv.org/abs/2210.02399) [cs.CV (https://arxiv.org/archive/cs.CV)].
99. Chang, Huiwen; Zhang, Han; Barber, Jarred; Maschinot, A. J.; Lezama, Jose; Jiang, Lu; Yang, Ming-Hsuan; Murphy, Kevin; Freeman, William T. (2023-01-02). "Muse: Text-To-Image Generation via Masked Generative Transformers". arXiv:2301.00704 (https://arxiv.org/abs/2301.00704) [cs.CV (https://arxiv.org/archive/cs.CV)].

100. Ramesh, Aditya; Pavlov, Mikhail; Goh, Gabriel; Gray, Scott; Voss, Chelsea; Radford, Alec; Chen, Mark; Sutskever, Ilya (2021-02-26). "Zero-Shot Text-to-Image Generation". arXiv:2102.12092 (https://arxiv.org/abs/2102.12092) [cs.CV (https://arxiv.org/archive/cs.CV)].
101. Yu, Jiahui; Xu, Yuanzhong; Koh, Jing Yu; Luong, Thang; Baid, Gunjan; Wang, Zirui; Vasudevan, Vijay; Ku, Alexander; Yang, Yinfei (2022-06-21). "Scaling Autoregressive Models for Content-Rich Text-to-Image Generation". arXiv:2206.10789 (https://arxiv.org/abs/2206.10789) [cs.CV (https://arxiv.org/archive/cs.CV)].
102. Kariampuzha, William; Alyea, Gioconda; Qu, Sue; Sanjak, Jaleal; Mathé, Ewy; Sid, Eric; Chatelaine, Haley; Yadaw, Arjun; Xu, Yanji; Zhu, Qian (2023). "Precision information extraction for rare disease epidemiology at scale" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9972634). *Journal of Translational Medicine*. **21** (1): 157. doi:10.1186/s12967-023-04011-y (https://doi.org/10.1186%2Fs12967-023-04011-y). PMC 9972634 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9972634). PMID 36855134 (https://pubmed.ncbi.nlm.nih.gov/36855134).
103. Hsu, Cyril Shih-Huan; Dalgkitsis, Anestis; Grosso, Paola; Papagianni, Chrysa (2026). "Transformer-Empowered Actor-Critic Reinforcement Learning for Sequence-Aware Service Function Chain Partitioning". *IEEE Transactions on Network Science and Engineering*. **13**: 9247–9264. arXiv:2504.18902 (https://arxiv.org/abs/2504.18902). Bibcode:2026ITNSE..13.9247H (https://ui.adsabs.harvard.edu/abs/2026ITNSE..13.9247H). doi:10.1109/TNSE.2026.3689920 (https://doi.org/10.1109%2FTNSE.2026.3689920).

Further reading

- Alexander Rush, The Annotated transformer (https://nlp.seas.harvard.edu/2018/04/03/attention.html) Archived (https://web.archive.org/web/20210922093841/https://nlp.seas.harvard.edu/2018/04/03/attention.html) 2021-09-22 at the Wayback Machine, Harvard NLP group, 3 April 2018
- US patent 10452978 (https://worldwide.espacenet.com/textdoc?DB=EPODOC&IDX=US10452978), Noam M. Shazeer; Aidan Nicholas Gomez; Lukasz Mieczyslaw Kaiser; Jakob D. Uszkoreit; Llion Owen Jones; Niki J. Parmar; Illia Polosukhin; Ashish Teku Vaswani, "Attention-based sequence transduction neural networks", issued 2019-10-22, assigned to Google LLC
- Phuong, Mary; Hutter, Marcus (2022). "Formal Algorithms for Transformers". arXiv:2207.09238 (https://arxiv.org/abs/2207.09238) [cs.LG (https://arxiv.org/archive/cs.LG)].

Retrieved from "https://en.wikipedia.org/w/index.php?title=Transformer_(deep_learning)&oldid=1382138706"